

CSCE 701 CYBERSECURITY · MODULE 01 · LECTURE NOTES

Thinking About Security

The asymmetry, and what follows from it.

MODULE OUTCOMES

- State the defender's asymmetry and its consequences.
- Build a threat model by a systematic method.
- Specify an adversary rather than assuming one.
- Explain defence in depth, least privilege, and assume-breach as responses.
- State the professional and legal constraints on this work.

1 The asymmetry

THE DEFENDER MUST COVER EVERYTHING; THE ATTACKER NEEDS ONE WAY IN

A system is secure only if every path into it is secure. It is compromised if any single path is not. So the defender faces a conjunction over all paths and the attacker faces a disjunction — and those are structurally different problems, with different effort profiles.

Which means security does not compose the way correctness does. Two correct components compose into a correct system; two secure components can compose into an insecure one — through their interaction, through shared state, or through the trust each places in the other. So a component-by-component review is not a system review, which is why Module 01's threat model works on the system diagram.

And it means the work is never finished, because the attacker's set of available paths grows as the system grows — every feature, dependency, integration, and configuration option adds to it — while the defender's review budget does not.

So the response is not 'be secure' but 'raise the attacker's cost, reduce the surface, and detect the failures' — which is precisely what defence in depth, least privilege, and assume-breach each do (§3). Every principle in this course is a response to this one paragraph.

Consequence	Why it follows	What to do about it
Security does not compose	Component interactions create paths that neither component has on its own.	Model the system and its trust boundaries, not the parts (§2).

Consequence	Why it follows	What to do about it
Absence of evidence is weak evidence	You cannot test for the absence of an attack path , only for the presence of specific ones.	Testing finds bugs rather than establishing security — CSCE 627 Module 07 §3's testing asymmetry, and see the note.
Complexity is the enemy	Attack surface grows faster than review capacity , and nobody reviews a configuration space.	Reduce the surface before adding controls — removing a feature is the strongest mitigation available.
The weakest link decides the outcome	The attacker chooses which link to attack, and will choose the weakest.	Find yours before they do , which means looking at the boring parts rather than the interesting ones.
And prevention will fail	Eventually, somewhere, for reasons you did not anticipate.	Design detection on the assumption that it has already happened (Module 09).

The second row is the most important, and it is the professional habit to acquire: 'we found no vulnerabilities' and 'there are no vulnerabilities' are completely different claims, and only the first is ever supportable. It is CSCE 627 Module 12's soundness distinction in a different setting — a security review is an incomplete search, and reporting it as a clean bill of health is the error that makes assessments useless.

2 Threat modelling

1. WHAT ARE WE BUILDING?

A diagram, with TRUST BOUNDARIES marked explicitly. Data flows that cross a boundary are where the threats are. Getting the boundaries wrong is the commonest failure in a threat model, and it is the graded part of this course's first project.

2. WHAT CAN GO WRONG?

Enumerate SYSTEMATICALLY, not by brainstorming.

STRIDE, applied to each data flow:

Spoofing	-> authentication
Tampering	-> integrity
Repudiation	-> logging, signatures
Information leak	-> confidentiality
Denial of service	-> availability
Elevation of priv.	-> authorisation

3. WHAT ARE WE GOING TO DO ABOUT IT?

Mitigate, eliminate (remove the feature entirely), transfer (to a provider or an insurer), or ACCEPT -- and accepting IN WRITING is a legitimate answer.

4. DID WE DO A GOOD JOB?

Review it, test that the mitigations work, and revisit it when the system changes.

THE VALUE IS IN STEP 2 BEING SYSTEMATIC. A brainstorm finds the threats you had already thought of; STRIDE applied to every flow finds the ones you had not.

SPECIFY THE ADVERSARY, BECAUSE DIFFERENT ADVERSARIES NEED DIFFERENT CONTROLS

'An attacker' is not a threat model. A bored stranger running a scanner, a motivated competitor, an organised criminal group, a current employee, a departing employee, and a state actor have entirely different capabilities, budgets, patience, and goals.

And the controls differ accordingly: rate limiting stops the first and not the last; an insider defeats your network boundary entirely (Module 04 §4) and must be addressed by authorisation, separation of duties, and logging instead (Module 03).

So write down, for each adversary you care about: their goal, their capability, their budget, and what access they already have. The last is the one that changes the model most — an adversary who already holds a valid credential is a different problem from one who does not.

And be realistic about which adversaries apply. Most systems are not state-actor targets, and designing against one while leaving default credentials in place is the common and embarrassing failure — a threat model should be proportionate to the assets and the plausible adversaries, and an implausible one consumes the budget that the plausible one needed.

3 The principles

- **Least privilege.** Give every component, process, and person the minimum access needed to do their job — so that a compromise yields less. **It reduces the *consequence* of a breach rather than its *probability***, which is worth being clear about because the two are frequently conflated.
- **Defence in depth.** Multiple independent controls, so that one failing is not a compromise — and 'independent' is the load-bearing word: three controls that all depend on the same identity provider, or all share an implementation flaw, are one control wearing three hats.
- **Assume breach.** Design for the case where an attacker is already inside the perimeter — which produces network segmentation (Module 04 §3), short-lived credentials (Module 07), and comprehensive logging (Module 09). **It is the principle that most changes an architecture.**
- **Fail closed.** An error condition should deny access rather than permit it — and getting this wrong is a frequent and severe class of bug: an authorisation check that throws an exception and is caught by a handler returning success, or a policy evaluation that defaults to allow when the policy service is unreachable.
- **And open design.** Security must not depend on the design being secret, because designs leak, are reverse engineered, and are discussed by former employees — whereas keys can be rotated (CSCE 711 Module 01 §2's Kerckhoffs principle). **Least privilege and defence in depth address different things — consequence and probability — and a programme with only one of them is incomplete.**

4 The constraints

AUTHORISATION IS NOT OPTIONAL, AND IT IS THE FIRST THING

Testing a system you are not authorised to test is unlawful in most jurisdictions, regardless of your intent, and 'I was only looking' is not a defence — unauthorised access statutes generally turn on access rather than on harm.

So: your own systems; your employer's systems with written authorisation and a defined scope; a bug bounty programme within its published rules; or a deliberately vulnerable training target designed for the purpose. Nothing else, ever, and the scope matters as much as the permission.

And if you find something unexpectedly — in a product you use, by accident, in the course of normal work — report it through a coordinated disclosure process and stop testing immediately rather than investigating further to characterise it. The instinct to confirm the finding is exactly the instinct to resist.

This course teaches how systems fail so that you can build ones that do not. Every technique here has a defensive purpose, and the material is scoped to that — which is the right scope for an engineer who will spend their career defending rather than attacking, and is not a limitation on what you can learn to do.

WHERE THIS COURSE GOES

Modules 02 through 06 are the controls. Authentication, authorisation, network security, platform isolation, and web security — **which is where the real failures are, and the ordering is roughly by how often each appears as the root cause in a published incident report.**

Modules 07 and 08 are secrets and the supply chain. Key management is the genuinely hard part of applied cryptography (CSCE 711 supplies the primitives, which work) **and dependencies are the fastest-growing attack surface in modern software.**

Modules 09 through 11 are detection, response, and people. What you do when prevention fails — and it will, which is §1's last row.

Modules 12 and 13 are measurement and honest claims — which is harder in this subject than in any other course in the program, because the thing you want to measure is the absence of something and §1's second row says you cannot.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Anderson — Security Engineering, chapters 1–3 (free PDF)	The asymmetry, the principles, and the economics — free in full, and the first chapter is the best framing of this subject in print.
Shostack — Threat Modeling, and the free four-question materials	§2's method , including STRIDE per element and the trust-boundary guidance.
Saltzer & Schroeder — The Protection of Information in Computer Systems (free)	§3's principles, in the 1975 original — least privilege, fail-safe defaults, and open design were all stated here, and the paper reads as current.
OWASP — Threat Modeling Cheat Sheet (free)	§2 as a working procedure , with the common mistakes listed.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Diagram a system you built** and mark every trust boundary. Count the data flows crossing one.
- 2 Apply STRIDE to each crossing flow** and record every threat, however unlikely.
- 3 Compare against a list you produce by brainstorming** and report how many the systematic method found that the brainstorm did not.

- 4 **Write adversary profiles** for three plausible adversaries of your system, with goal, capability, budget, and existing access.
- 5 **Identify a control that stops one adversary and not another**, and say why.
- 6 **For each threat, decide: mitigate, eliminate, transfer, or accept** — and write the acceptances down.
- 7 **Find two of your controls that are not independent**, and say what they share.
- 8 **Find a place where your system fails open** and fix it.
- 9 **Look up the unauthorised-access law in your jurisdiction** and summarise what it prohibits.
- 10 **Find the coordinated disclosure policy** for a product you use, and note whether one exists.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the asymmetry and explain why security does not compose.
- 2 Why are 'found no vulnerabilities' and 'there are none' different claims?
- 3 Give five consequences of the asymmetry.
- 4 Give the four threat-modelling questions and the STRIDE categories.
- 5 Why must enumeration be systematic?
- 6 Why is 'an attacker' not a threat model, and what should you write instead?
- 7 Name five principles and say what each addresses.
- 8 Which reduce probability and which reduce consequence?
- 9 What are the constraints on security testing, and what do you do on an accidental finding?